A Dual-Branch Deep Stochastic Adaptive Fourier Decomposition Network for Hyperspectral Image Classification

Chunbo Cheng, Liming Zhang, *Member, IEEE*, Hong Li, *Member, IEEE*, and Wenjing Cui

***Abstract*— Recently, hyperspectral image (HSI) classification methods based on deep learning (DL) have demonstrated excellent performance. However, these DL methods still face two major challenges. One is that they require a large number of labeled samples, and the other is that training parameters take a lot of time. In this article, we propose a dual-branch deep stochastic adaptive Fourier decomposition (SAFD) network (DSAFDNet) to alleviate the aforementioned two issues in HSI classification applications. SAFD is a newly developed signal processing tool with a solid mathematical foundation. It can be used to find common filters (i.e., convolution kernels) of a set of random signals (RSs) or multisignals. Since the convolution kernels obtained by SAFD decomposition are complex numbers, few DL methods directly deal with such complex convolution kernels. To this end, we propose a dual-branch network to extract deep features from HSIs using both real and imaginary parts of convolutional kernels. After deep feature extraction using DSAFDNet, we further investigate the classification performance of different classifiers on the extracted features. Experimental results show that the proposed method outperforms some HSI classification methods with similar principles. Moreover, compared with other state-of-the-art DL methods, the proposed method can achieve better classification performance.**

***Index Terms*— Deep learning (DL), graph-based learning (GSL), hyperspectral image (HSI) classification, stochastic adaptive Fourier decomposition (SAFD).**

I. Introduction

HYPERSPECTRAL image (HSI) is presented in the form of the data cube. It not only contains spatial features but also provides a continuous diagnostic spectrum, which can enhance the characterization and determination of ground objects [1]. HSI classification is a significant task in the remote sensing application area that assigns different regions with labels based on the multiple dimensional feature space [2]. In the last two decades, many methods were introduced for the HSI classification [3], [4]. For example, Li et al. [5] proposed a framework called multiscale spatial information fusion (MSIF), which consists of multiscale spatial information extraction, local 1-D [6] embedding using spectral information and spatial information at different scales, and information fusion. Gao et al. [7] proposed a spectral and spatial classification framework for HSI, which uses random multigraphs (RMGs). However, these methods are restricted since they only extract low-level features and ignore high-level features, thus failing to achieve higher accuracies. Meanwhile, the performance of these methods depends on handcrafted features, which handcrafted features are regarded as features that lack strong robustness in the face of complex situations [4].

To overcome this shortcoming, deep learning (DL) [3], [4] with powerful representation capabilities emerged and made great progress in HSI classification. Compared with the traditional shallow classification model [3], the DL model is regarded as a classification model with a multilayer structure, which can extract rich spatial-spectral information automatically. A general framework of HSI classification based on DL can be divided into two modules [4]: deep features extraction and classification. The classification accuracy of a classifier largely depends on the extracted features [8]. There are various deep feature extraction models in the literature.

Convolutional neural networks (CNNs) are increasingly used for HSI classification [9], [10], [11]. These methods either use only spectral information or spatial information [12]. In order to combine spatial and spectral information, Roy et al. [13] first used 3-D-CNN to extract spatial–spectral features, and then used 2-D-CNN to further extract spatial features. Yu et al. [12] used the residual neural network (ResNet) [14] as the backbone network and proposed a dual-branch ResNet (DBRNet) to fuse the spectral information and spatial information. Hao et al. [15] proposed a two-stream network, where one stream uses a stacked denoising autoencoder to learn the spectral features of a certain pixel and the other uses a 2-D CNN to learn the spatial features surrounding the pixel. A novel approach in [16] was proposed to boost the network representation power with a two-stream 2-D CNN architecture, which extracts simultaneously the spectral features, local

Manuscript received 1 January 2023; revised 6 May 2023 and 13 July 2023; accepted 14 August 2023. Date of publication 22 August 2023; date of current version 8 September 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62206088 and Grant 61877021, in part by the Hubei Polytechnic University under Grant 21xjz25R, in part by the Science and Technology Development Fund of Macau SAR under Grant 0123/2018/A3 and Grant 0060/2021/A, and in part by the Multi-Year Research Grant under Grant MYRG2022-00193-FST. *(Corresponding author: Liming Zhang.)*

Chunbo Cheng and Wenjing Cui are with the School of Mathematics and Physics, Hubei Polytechnic University, Huangshi 435000, China (e-mail: bccheng@hbpu.edu.cn; wjcui@hbpu.edu.cn).

Liming Zhang is with the Faculty of Science and Technology, University of Macau, Macau, China (e-mail: lmzhang@um.edu.mo).

Hong Li is with the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: hongli@hust.edu.cn).

Digital Object Identifier 10.1109/TGRS.2023.3307598

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

spatial and global spatial features. The multibranch CNN method based on adaptive region search (RS-AMCNN) in [17] was proposed for HSI classification, which can extract joint spatial-spectral features in each branch. Yu et al. [2] presented a spatial–spectral dense CNN framework with a feedback attention mechanism for HSI classification, which assembles the spectral–spatial feature in a compact connection style to extract sufficient information independently with two separate dense CNN networks. A dual-channel spectral–spatial fusion capsule generative adversarial network (DcCapsGAN) was built in [18] for HSI classification and multiscale convolution was integrated into DcCapsGAN to extract multiscale features.

Although these DL methods have achieved satisfactory results, there are still some issues worth considering: 1) since the number of channels of HSI data is large, and such complex data types usually cause more network parameters [12]; 2) training a large network generally requires a large number of labeled samples, but there are usually fewer labeled samples and a large number of unlabeled samples in HSI data [12]; and 3) DL methods usually take a lot of time to train network parameters [4]. Therefore, constructing a new DL method needs to solve the following two problems [12]. First, how to jointly use spatial and spectral information, and second, how to train a classification network with a small number of labeled samples and a large number of unlabeled samples [12].

Recently, a new sparse representation method, adaptive Fourier decomposition (AFD) [19], [20] is developed, which integrates the advantages of transformation-based and learning-based sparse representations. There are several variants of AFD, and stochastic AFD (SAFD) is one of them. SAFD is developed to process multisignals with the same atoms (i.e., the entry in a dictionary is called an atom, and it is a vector in a dictionary), which has been successfully used in image compression [21], video compression [22] and electrocardiogram (ECG) based personal identification [23]. However, these methods focus on learning shallow features.

In this article, we focus on the development of an SAFD-based deep network that can learn deep features. The coefficients produced by SAFD when decomposing a signal are complex numbers. The current DL methods can basically only process signals or data in the real number domain, but cannot process signals or data directly in the complex number domain. In this article, we propose to use dual-branches to fuse two kinds of information extracted from real and imaginary parts of complex coefficients. Furthermore, since the classification accuracy of a classifier depends largely on the extracted features, we also investigate the classification performance of different classifiers on the features extracted by the proposed deep SAFD network.

The main contributions and novelty of this article are summarized as follows.

1) A dual-branch deep SAFD network (DSAFDNet) is proposed, where the dual branch is used to extract features from HSIs using the real and imaginary parts of complex convolutional kernels trained by SAFD, respectively. Since SAFD itself can automatically learn the common features of multiple signals, the deep network based on SAFD can also obtain higher HSI classification performance in the case of a small number of samples.
2) The proposed DSAFDNet combines deep feature extraction with a classifier without using back propagation and fine-tuning, thus being unsupervised learning and saving training time. Furthermore, the same dual branch is used for both spectral and spatial feature extraction. Experiments show that the proposed method achieves better performance compared to other selected state-of-the-art methods.
3) We investigate the classification performance of different classifiers on the features extracted by the proposed DSAFDNet. Specifically, we compare the performance of DSAFDNet classification with two popular classifiers support vector machine (SVM) and graph-based learning (GSL) method [4] on three real hyperspectral datasets, respectively. The experimental results show that the classifier GSL can achieve better classification performance among all comparison methods.

The remainder of this article is organized as follows. Section II introduces the preliminary knowledge of SAFD. Section III-A presents the proposed SAFD model for HSI, which is designed to learn convolution kernels from HSIs and lays the theoretical foundation for the DSAFDNet. Sections III-B–III-D present DSAFDNet for deep feature extraction, computational complexity of DSAFDNet and the classification framework of HSI, respectively. Sections IV and V are experiments and conclusion, respectively.

II. Preliminary

Since the proposed deep network is based on SAFD theory, this section will briefly introduce the basic theory of SAFD. It has appeared in the literature [19], [20], [23].

A. Preliminary Knowledge of SAFD

Suppose that $f^i(i = 1, 2, \ldots, N)$ is a group of analytic random signals (RSs) in Hardy space $H^2(\mathbb{D})$ from which SAFD can be used to train a common set of atoms. $\mathbb{D}$ is the unit disk in the complex plane. The first step of the SAFD is

$$f^i(z) = f_1^i(z) = \left(f_1^i(z) - \left\langle f_1^i, e_{a_1}\right\rangle e_{a_1}(z)\right) + \left\langle f_1^i, e_{a_1}\right\rangle e_{a_1}(z) = f_2^i(z)\frac{z - a_1}{1 - \bar{a}_1 z} + \left\langle f_1^i, e_{a_1}\right\rangle \mathcal{B}_{a_1}(z) \tag{1}$$

where

$$f_2^i(z) = \frac{f_1^i(z) - \left\langle f_1^i, e_{a_1}\right\rangle e_{a_1}(z)}{\frac{z-a_1}{1-\bar{a}_1 z}} \quad \forall a_1 \in \mathbb{D}. \tag{2}$$

Repeating the above process up to n-times, it can obtain

$$f^i(z) = f_{n+1}^i(z)\prod_{k=1}^{n}\frac{z - a_k}{1 - \bar{a}_k z} + \sum_{k=1}^{n}\left\langle f_k^i, e_{a_k}\right\rangle \mathcal{B}_k(z) \tag{3}$$

where

$$\mathcal{B}_k(z) = e_{a_k}(z)\prod_{l=1}^{k-1}\frac{z - a_l}{1 - \bar{a}_l z}. \tag{4}$$

$\{\mathcal{B}_k\}_{k=1}^{\infty}$ is the automatically generated orthonormal system corresponding to the maximally selected $a_1, a_2, \ldots, a_k \in \mathbb{D}$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

The reduced remainders $f_{k-1}^i(z)$ $(k = 2, 3, \ldots, n, \ldots)$ are defined by

$$f_k^i(z) = \frac{\left(f_{k-1}^i(z) - \left\langle f_{k-1}^i, e_{a_{k-1}}\right\rangle e_{a_{k-1}}(z)\right)}{\frac{z-a_k}{1-\bar{a}_k z}} \in H^2(\mathbb{D}). \quad (5)$$

We have chosen sequence a_k at each step that give rise to the best approximation in the probabilistic sense. The maximal selection principle (MSP) is used to solve the feasibility of an optimal selection of the parameter. Based on the orthogonalization of (1), there is

$$a_1 = \arg\max\left\{E_i\left|\left\langle f_1^i, e_a\right\rangle\right|^2, a \in \mathbb{D}\right\} \quad (6)$$

where E_i represents mathematical expectation. The function e_a is called the evaluator at $a \in \{a_1, a_2, \ldots, a_k\}_{k=1}^{\infty}$, and each evaluator gives rise to an evaluating functional. Summing over (6) with respect to i, it can obtain

$$E_i\left|\left\langle f_1^i, e_a\right\rangle\right|^2 = \frac{1}{N}\sum_{i=1}^{N}\left|\left\langle f_1^i, e_a\right\rangle\right|^2. \quad (7)$$

a_k can be obtained by the consecutive stochastic MSP (SMSP), which is

$$a_k = \arg\max\left\{E_i\left|\left\langle f_k^i, e_a\right\rangle\right|^2, a \in \mathbb{D}\right\} \quad (8)$$

where

$$E_i\left|\left\langle f_k^i, e_a\right\rangle\right|^2 = \frac{1}{N}\sum_{i=1}^{N}\left|\left\langle f_k^i, e_a\right\rangle\right|^2. \quad (9)$$

Literature [24] provided the existence of a_k and the convergence proof of the formula.

B. Implementation Process of SAFD

SAFD can directly decompose the original multiple RSs $\{f^i(z)\}_{i=1}^N$, and can rapidly converge. SAFD is used to decompose the multiple RSs to obtain a universal dictionary and the coefficient matrix. Suppose m is the number of sampling points for each RS, and C is the decomposition number. Then the dictionary $\mathcal{D}$ can be defined as follows:

$$\mathcal{D} = [\mathcal{D}_1, \ldots, \mathcal{D}_j, \ldots, \mathcal{D}_C] \quad (10)$$

where

$$\mathcal{D}_j = \begin{pmatrix} \mathcal{B}_{\{a_1,\ldots,a_j^1\}}(e^{it_1}) & \cdots & \mathcal{B}_{\{a_1,\ldots,a_j^L\}}(e^{it_1}) \\ \vdots & \vdots & \vdots \\ \mathcal{B}_{\{a_1,\ldots,a_j^1\}}(e^{it_m}) & \cdots & \mathcal{B}_{\{a_1,\ldots,a_j^L\}}(e^{it_m}) \end{pmatrix} \quad (11)$$

where $a_k \in [a_k^1, a_k^2, \ldots, a_k^L]$, and the number of atoms in each decomposition level is L. $\mathcal{D}$ can be calculated by (4) from the original signal, and its size is $m \times CL$. Once $\mathcal{D}$ is obtained, the coefficient matrix can be calculated by

$$W = \begin{pmatrix} \mathcal{W}_{11} & \cdots & \mathcal{W}_{1i} & \cdots & \mathcal{W}_{1N} \\ \vdots & & \vdots & & \vdots \\ \mathcal{W}_{k1} & \cdots & \mathcal{W}_{ki} & \cdots & \mathcal{W}_{kN} \\ \vdots & & \vdots & & \vdots \\ \mathcal{W}_{C1} & \cdots & \mathcal{W}_{Ci} & \cdots & \mathcal{W}_{CN} \end{pmatrix} \quad (12)$$

where

$$\mathcal{W}_{ki} = \begin{pmatrix} \left\langle f_k^i, e_{a_k^1}\right\rangle \\ \vdots \\ \left\langle f_k^i, e_{a_k^j}\right\rangle \\ \vdots \\ \left\langle f_k^i, e_{a_k^L}\right\rangle \end{pmatrix}. \quad (13)$$

$\mathcal{W} \in \mathbb{C}^{CL\times N}$ ($\mathbb{C}$ is the field of complex numbers) and f_k^i is calculated by (5).

III. Proposed Method

Recently, HSI classification based on DL has achieved great success, especially CNN has attracted extensive attention. For CNN, its two important components are deep structure and convolution kernel [4], in which deep structure can extract deep and abstract features, and convolution kernel can represent the spatial structure of images. The convolution kernels can also be regarded as weights, filters, or coefficients. In Section II-B, SAFD can be decomposed to obtain the dictionary and coefficient matrix. The coefficient matrix can be used to get convolution kernels or filters.

In this section, we first introduce SAFD to decompose HSIs to obtain coefficient matrices (i.e., convolution kernels). We then show how to use this coefficient matrix to build DL architectures.

A. Learning Convolution Kernels From HSIs Using SAFD

An HSI of size $w \times h \times q$ is divided into nonoverlapping image patches of the same size, then SAFD can be used to decompose these image patches to obtain a universal convolution kernel. If the size of each image patch is $w_0 \times h_0 \times q$, then there are (wh/w_0h_0) image patches. SAFD can handle 1-D signals, but these image patches are multidimensional, so they need to be processed in order to be decomposed using SAFD. The specific steps are as follows.

For one image patch, a certain channel c $(c = 1, 2, \ldots, q)$ of the image patch is converted into a 1-D RS, denoted as g_c^i $[i = 1, 2, \ldots, (wh/w_0h_0)]$, then each RS with the decomposition level C can be decomposed as

$$g_c^i(z) = \sum_{j=1}^{C}\frac{1}{\alpha}\left\lfloor\alpha\left\langle g_{c,j}^i, e_{a_{c,j}}\right\rangle\right\rfloor\mathcal{B}_{c,j}(z) \quad (14)$$

where

$$\mathcal{B}_{c,j}(z) = e_{a_{c,j}}(z)\prod_{m=1}^{k-1}\frac{z-a_{c,m}}{1-\bar{a}_{c,m}z} \quad (15)$$

where "$\lfloor\ \rfloor$" meaning rounding to infinite can be defined as the floor function and α is the parameter for the quantization of coefficients. $a_{c,j}$ can be computed by SMSP

$$a_{c,j} = \frac{1}{\rho}\left\lfloor\rho\arg\max\left\{E_i\left|\left\langle g_{c,j}^i, e_a\right\rangle\right|^2, a \in \mathbb{D}\right\}\right\rfloor \quad (16)$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

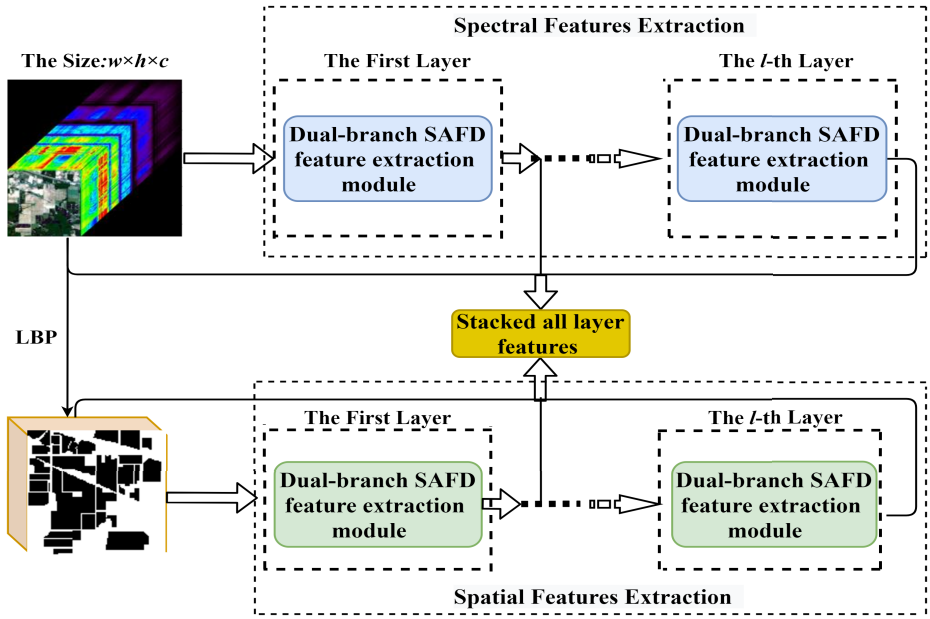

Fig. 1. Schematic of DSAFDNet structure for HSI feature extraction. It contains the same feature extraction module at each layer, i.e., dual-branch SAFD feature extraction module. The detailed architecture of the dual-branch SAFD feature extraction module is shown in Fig. 2.

where ρ is the parameter used for quantization of $a_{c,j}$, and $g^i_{c,j}(z)$ is recursively obtained by

$$g^i_{c,j}(z) = \left(g^i_{c,j-1}(z) - \frac{1}{\alpha} \lfloor \alpha \langle g^i_{c,j-1}, e_{a_{c,j-1}} \rangle \rfloor e_{a_{c,j-1}}(z) \right) \beta_{c,j-1}(z) \quad \forall c \in \{1, 2, \ldots, q\} \tag{17}$$

where

$$\beta_{c,j-1}(z) = \frac{1 - \bar{a}_{c,j-1} z}{z - a_{c,j-1}} \quad \forall c \in \{1, 2, \ldots, q\}. \tag{18}$$

Multidimensional image patches can be processed channel by channel. Algorithm 1 summarizes the training process of SAFD, abbreviated as SAFD_ker algorithm, which is used to learn the convolution kernel.

B. Architecture of Dual-Branch Deep SAFD Network for Deep Feature Extraction

Inspired by the structure of deep CNN, we constructed a deep SAFD network with a dual-branch for deep feature extraction of HSIs, as shown in Fig. 1. Different from CNN, the DSAFDNet we built has no pooling layer, batch standardization layer, and fully connected layer. In addition, there is no back propagation and fine-tuning process, that is, DSAFDNet can efficiently extract deep features without back propagation and fine-tuning in the feature extraction stage. Then, the extracted deep features are classified using GSL method and SVM in order to study the classification performance of different classifiers.

1) Extracting Deep Spectrum Features Using DSAFDNet: Based on SAFD, we construct a deep network with dual-branch, as shown in Fig. 2. In DSAFDNet, we use two networks with identical submodules (see Fig. 2) to extract spectral and spatial information of HSIs, respectively. Each module is a shallow SAFD with dual-branch, which includes principal component analysis (PCA), whitening, convolution, and rectified linear units (ReLu) operations.

Algorithm 1 SAFD_ker Learning Algorithm

Input: An HSI $\mathcal{Y}$ with $w \times h \times q$; The size of image patches $w_0 \times h_0 \times q$; The decomposition level C.
Output: Convolution kernel $\mathcal{W} \in \mathbb{C}^{CL \times N}$.

Number of input signals or image patches $s = \frac{wh}{w_0 h_0}$.
for c=1:q **do**
  for i=1:N **do**
    Convert image patches to RSs $\{g^i_c(z)\}_{i=1}^N$;
    $\hat{g}^i_{c,1}(z) = g^i_c(z)$;
  end for
  Calculate $a_{c,1}$ using equation (16).
end for
for c=1:q **do**
  for j=2:C **do**
    for i=1:N **do**
      $\beta_{c,j-1}(z) = \frac{1-\bar{a}_{c,j-1}z}{z-a_{c,j-1}}$;
      $v^i_{c,j-1}(z) = \frac{1}{\alpha} \lfloor \alpha \langle \hat{g}^i_{c,j-1}, e_{a_{c,j-1}} \rangle \rfloor e_{a_{c,j-1}}(z)$;
      $\hat{g}^i_{c,j}(z) = (\hat{g}^i_{c,j-1}(z) - v^i_{c,j-1}(z))\beta_{c,j-1}(z)$;
      Convolution kernel $\mathcal{W}^i_{c,j} = \lfloor \alpha \langle \hat{g}^i_{c,j}, e_{a_{c,j-1}} \rangle \rfloor$;
      Calculate $\mathcal{W}$ using equation (12) and (13);
    end for
    Calculate $a_{c,j}$ using equation (16).
  end for
end for
Return: Convolution kernel $\mathcal{W}$.

The convolution kernel learned using Algorithm 1 in Section II-B is a complex number. Current deep networks

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

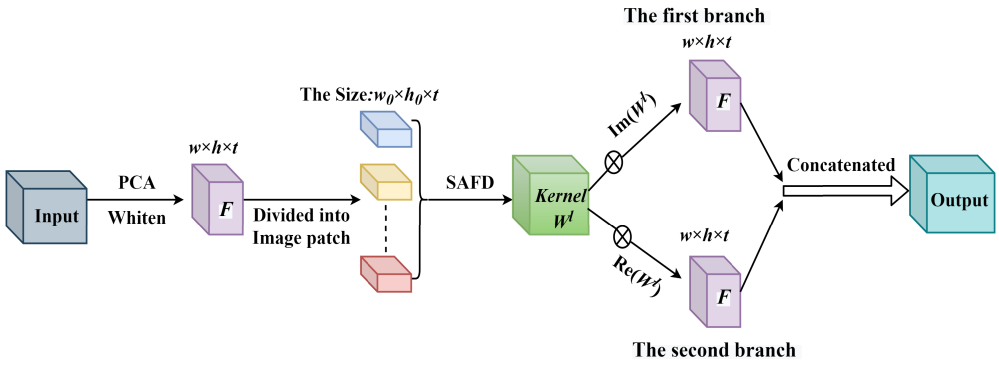

Fig. 2. Dual-branch SAFD feature extraction module.

rarely deal with complex numbers. In order to make full use of the real and imaginary parts of $\mathcal{W}$, we introduce dual-branch. Specifically, the real part and imaginary part of $\mathcal{W}$ are regarded as two convolution kernels, and the features of HSIs are simultaneously extracted by the real part and imaginary part.

Let $\mathcal{Y} \in \mathbb{R}^{w\times h\times q}$ be the original HSI data. Since the operations on each module are the same, we can deduce the mathematical expression for the features of any layer. Let $\mathcal{W}^\ell$ be the convolution kernel in the (ℓ)th layer, $\mathrm{Re}(\mathcal{W}^\ell)$ and $\mathrm{Im}(\mathcal{W}^\ell)$ denote its real and imaginary parts, respectively. Then the features of HSIs are simultaneously extracted by $\mathrm{Re}(\mathcal{W}^\ell)$ and $\mathrm{Im}(\mathcal{W}^\ell)$, and finally these two features are fused. Let $\boldsymbol{F}^{\ell-1}$ be the input of the ℓth layer, use $\mathrm{Re}(\mathcal{W}^\ell)$ and $\mathrm{Im}(\mathcal{W}^\ell)$ to convolve with $\boldsymbol{F}^{\ell-1}$, respectively, to obtain $\boldsymbol{F}_r^\ell$ and $\boldsymbol{F}_I^\ell$, and concatenate $\boldsymbol{F}_r^\ell$ and $\boldsymbol{F}_I^\ell$ to obtain the output $\boldsymbol{F}^\ell$ in the ℓth layer. Fig. 2 shows the workflow of SAFD feature extraction module. The specific mathematical representation is as follows.

Since the input $\boldsymbol{F}^{\ell-1}$ is a high-dimensional data, we first use PCA and whitening on $\boldsymbol{F}^{\ell-1}$, i.e.,

$$\hat{\boldsymbol{F}}^\ell = \textbf{Whiten_PCA}(\boldsymbol{F}^{\ell-1}, t) \tag{19}$$

where the notation $\textbf{Whiten_PCA}(\boldsymbol{F}^{\ell-1}, t)$ means using PCA on $\boldsymbol{F}^{\ell-1}$ and retaining the first t principal components, followed by whitening. Then, $\hat{\boldsymbol{F}}^\ell$ layer is divided into nonoverlapping image patches $\{\mathcal{P}_i^\ell\}_{i=1}^s$ of size $w_0 \times h_0 \times t$, where the size of each image patch is $w_0 \times h_0 \times q$, and there are $s = (wh/w_0h_0)$ image patches. SAFD is used to learn a convolution kernel $\mathcal{W}^\ell$ at the ℓth layer from a set of multidimensional image patches (see Algorithm 1), which is represented as

$$\mathcal{W}^\ell = \textbf{SAFD_ker}\left(\left\{\mathcal{P}_i^\ell\right\}_{i=1}^s\right) \in \mathbb{R}^{tw_0h_0\times K}. \tag{20}$$

2) Feature Extracted by Real Part $Re(\mathcal{W}^\ell)$: Assume $\boldsymbol{F}_r^{\ell-1}(\ell \geq 2)$ be the feature extracted by $\mathrm{Re}(\mathcal{W}^\ell)$ in the $(\ell-1)$th layer, $\boldsymbol{F}_r^{\ell-1}$ is used as the input of the ℓth layer, then the output $\boldsymbol{F}_r^\ell$ of the ℓth layer is

$$\begin{aligned}
&\boldsymbol{F}_j^\ell = \sum_{j=1}^{K} \hat{\boldsymbol{F}}_j^\ell \otimes \mathrm{Re}\left(\mathcal{W}_{ij}^\ell\right) \quad \forall i = 1, 2, \ldots, K \\
&\boldsymbol{F}_r^\ell = \max\left(0, \boldsymbol{F}_r^\ell - \boldsymbol{M}_r^\ell\right), \quad \hat{\boldsymbol{F}}_j^\ell \in \hat{\boldsymbol{F}}^\ell,\ \boldsymbol{F}_j^\ell \in \boldsymbol{F}_r^\ell
\end{aligned} \tag{21}$$

where $\boldsymbol{M}_r^\ell \in \mathbb{R}^{wh\times K}$ is composed of K replications of the mean matrix of $\boldsymbol{F}_r^\ell \in \mathbb{R}^{wh\times K}$. "$\otimes$" is a 2-D convolution operation.

3) Feature Extracted by Imaginary Part $Im(\mathcal{W}^\ell)$: Keeping the input unchanged, the imaginary part is selected as the convolution kernel for feature extraction. Assume $\boldsymbol{F}_I^{\ell-1}(\ell \geq 2)$ be the feature extracted by $\mathrm{Im}(\mathcal{W}^\ell)$ in the $(\ell-1)$th layer, $\boldsymbol{F}_I^{\ell-1}$ is used as the input of the ℓth layer, then the output $\boldsymbol{F}_I^\ell$ of the ℓth layer is

$$\begin{aligned}
&\boldsymbol{F}_j^\ell = \sum_{j=1}^{K} \hat{\boldsymbol{F}}_j^\ell \otimes \mathrm{Im}\left(\mathcal{W}_{ij}^\ell\right) \quad \forall i = 1, 2, \ldots, K \\
&\boldsymbol{F}_I^\ell = \max\left(0, \boldsymbol{F}_I^\ell - \boldsymbol{M}_I^\ell\right), \quad \hat{\boldsymbol{F}}_j^\ell \in \hat{\boldsymbol{F}}^\ell,\ \boldsymbol{F}_j^\ell \in \boldsymbol{F}_I^\ell
\end{aligned} \tag{22}$$

where $\hat{\boldsymbol{F}}_I^\ell$ in the ℓth layer is divided into nonoverlapping image patches $\{\mathcal{P}_i^\ell\}_{i=1}^s$ with size $w_0 \times h_0 \times t$. $\boldsymbol{M}_I^\ell \in \mathbb{R}^{wh\times K}$ is composed of K replications of the mean matrix of $\boldsymbol{F}_I^\ell \in \mathbb{R}^{wh\times K}$.

Finally, a fused feature $\boldsymbol{F}^\ell = [\boldsymbol{F}_r^\ell \ \ \boldsymbol{F}_I^\ell]$ in the ℓth layer is obtained by concatenating features $\boldsymbol{F}_r^\ell$ and $\boldsymbol{F}_I^\ell$. This is the whole SAFD feature extraction process.

Extracting deep spatial features using DSAFDNet, we use the same module with different input. The input is obtained by using the local binary pattern (LBP), which can be defined as

$$\mathcal{X} = \sum_{j=0}^{d^2-1} \mathcal{S}(P_j - P_c)2^j \tag{23}$$

where

$$\begin{cases}
\mathcal{S}(P_j - P_c) = 1, & \text{if} \quad P_j > P_c \\
\mathcal{S}(P_j - P_c) = 0, & \text{else if} \quad P_j \leq P_c
\end{cases} \tag{24}$$

where P_c represents local region centered at c, and its size is $d \times d$. In this way, features are extracted layer by layer using the same module (i.e., SAFD feature extraction module), namely, the features of each layer can be calculated using (19)–(22).

Finally, we stack the features of all layers, LBP feature $\mathcal{X}$ and original HSI data $\mathcal{Y}$ to obtain the combined feature $\mathcal{F}_{\text{fus}}$.

C. Complexity Analysis of Feature Extraction

Since DSAFDNet uses the same module (i.e., SAFD feature extraction module), the overall computational complexity can

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPUTATION COMPLEXITY OF DIFFERENT OPERATION IN DSAFDNET

Methods	Computational complexity
PCA	$\mathcal{O}(whq^2+q^3+whqt)$
SAFD algorithm	$\mathcal{O}(s^2)$
LBP	$\mathcal{O}(whqd^2-whq)$
Convolution	$\mathcal{O}(stwhw_0h_0+swht)$
ReLu	$\mathcal{O}(3whs)$
SAFD feature module	$\mathcal{O}(2stwh(w_0h_0+1)+wh(q^2+qt+6s)+q^3+s^2)$

be deduced only by analyzing the computational complexity of the SAFD feature extraction module. For the SAFD feature extraction module, PCA was first used to reduce the dimensionality of the original HSI $\mathcal{Y}\in\mathbb{R}^{w\times h\times q}$ and reserve the first t principal components, whose computational complexity is $\mathcal{O}(whq^2+q^3+whqt)$. Then, SAFD is used to learn a convolution kernel, whose computational complexity is $\mathcal{O}(s^2)$. Afterward, the real and imaginary parts of the trained convolution kernel are used to perform convolution operations with the input data, respectively, and then the output is obtained by concatenating the convolution results of the real and imaginary parts through the activation function. The computational complexity of the convolution operation and ReLu are $\mathcal{O}(stwhw_0h_0+swht)$ and $\mathcal{O}(3whs)$, respectively. Therefore, it can be concluded that the overall computational complexity of the SAFD feature extraction module in the worst case is

$$\mathcal{O}(2stwh(w_0h_0+1)+wh(q^2+qt+6\ s)+q^3+s^2). \quad (25)$$

When extracting spatial information, we first use LBP, and then also use the SAFD feature extraction module. The computational complexity of LBP is $\mathcal{O}(whqd^2-whq)$. Finally, the overall computational complexity of DSAFDNet can be obtained by summing the computational complexity of SAFD feature extraction modules on all layers. Table I shows that computational complexity of different methods in the DSAFDNet. As we can see, the main computational burdens come from the PCA and convolution operations. Subsequent experiments will give the running time of DSAFDNet.

D. Classification for HSI

In this section, we will classify the extracted combined features $\mathcal{F}_{\text{fus}}$. Specifically, the combined feature $\mathcal{F}_{\text{fus}}$ is obtained by combining the features on all layers of DSAFDNet, original HSI data $\mathcal{Y}$ and LBP feature $\mathcal{X}$.

Before classifying $\mathcal{F}_{\text{fus}}$, we first reshape the 3-D data $\mathcal{F}_{\text{fus}}\in\mathbb{R}^{w\times h\times(2q+4\ell K)}$ into a 2-D matrix $\bar{\mathcal{F}}_{\text{fus}}\in\mathbb{R}^{wh\times(2q+4\ell K)}$, and normalize it

$$\hat{\mathcal{F}}^i_{\text{fus}}=\frac{\bar{\mathcal{F}}^i_{\text{fus}}-\text{mean}\left(\bar{\mathcal{F}}^i_{\text{fus}}\right)}{\text{var}\left(\bar{\mathcal{F}}^i_{\text{fus}}\right)}\quad \forall\ i\in\{1,2,\ldots,2q+4\ell K\} \quad (26)$$

where $\hat{\mathcal{F}}^i_{\text{fus}}\in\mathbb{R}^{wh\times 1}$ denotes the normalized data, mean$(\cdot)$ and var$(\cdot)$ represent the mean and variance, respectively.

Here, we would like to further investigate the classification performance of different classifiers for $\hat{\mathcal{F}}_{\text{fus}}$. We select GSL in [4] and SVM in [25] to classify $\hat{\mathcal{F}}_{\text{fus}}$, respectively. The two classification algorithms are as follows.

1) Classification Based on GSL: First, $\hat{\mathcal{F}}_{\text{fus}}$ is divided into training samples $\mathcal{X}_{\text{tr}}=[\boldsymbol{x}_1,\boldsymbol{x}_2,\ldots,\boldsymbol{x}_l]$ and test samples $\mathcal{X}_{\text{tes}}=[\boldsymbol{x}_{l+1},\boldsymbol{x}_{l+2},\ldots,\boldsymbol{x}_M]$. GSL can be solved by the following quadratic optimization problem with k-class:

$$\min_{\mathcal{T}}\text{tr}(\mathcal{T}-\mathcal{V})^T\mathcal{M}(\mathcal{T}-\mathcal{V})+\text{tr}(\mathcal{T}^T\mathcal{G}\mathcal{T}) \quad (27)$$

where tr$(\cdot)$ is the trace function, $\mathcal{T}\in\mathbb{R}^{M\times k}$ indicates the label to be predicted

$$\mathcal{V}=\begin{pmatrix} v_{11} & \cdots & v_{1k}\\ \vdots & \ddots & \vdots\\ v_{M1} & \cdots & v_{Mk}\end{pmatrix},\quad \begin{cases} v_{ij}=1, & \text{if}\ \ \boldsymbol{x}_i\in j\\ v_{ij}=0, & \text{otherwise.}\end{cases} \quad (28)$$

$\mathcal{M}\in\mathbb{R}^{M\times M}$ is defined as follows:

$$\mathcal{M}=\begin{pmatrix}\theta_1 & \cdots & 0\\ \vdots & \ddots & \vdots\\ 0 & \cdots & \theta_M\end{pmatrix},\quad \begin{cases}\theta_i=\delta_n, & \text{if}\ \ 1\le i\le n\\ \theta_i=\delta_{M-n}, & \text{otherwise}\end{cases} \quad (29)$$

where parameters δ_n and δ_{M-n} need to be preset with initial values. $\mathcal{G}\in\mathbb{R}^{M\times M}$ is graph Laplacian, which is calculated by

$$\mathcal{G}=\mathcal{L}-\mathcal{Z} \quad (30)$$

where $\mathcal{Z}=\{\kappa_{ij}\}(i,j=1,\ldots,M)\in\mathbb{R}^{M\times M}$, and κ_{ij} is defined as follows:

$$\kappa_{ij}=e^{-\frac{\|x_i-x_j\|^2}{2\vartheta^2}}(i\ne j),\quad \kappa_{ii}=0 \quad (31)$$

where ϑ is the preset parameter, $\mathcal{L}$ is defined by

$$\mathcal{L}=\begin{pmatrix}\phi_1 & \cdots & 0\\ \vdots & \ddots & \vdots\\ 0 & \cdots & \phi_M\end{pmatrix},\quad \phi_i=\sum_{j=1}^{M}\kappa_{ij}. \quad (32)$$

2) Classification Based on SVM: Given a training set $\mathcal{H}=\{(\boldsymbol{x}_1,y_1),(\boldsymbol{x}_2,y_2),\ldots,(\boldsymbol{x}_l,y_l)\}\in\mathbb{R}^n\{-1;1\}$, and the decision function is found by solving the convex optimization problem

$$\max_{\alpha} g(\alpha)=\sum_{i=1}^{l}\alpha_i-\frac{1}{2}\sum_{i,j=1}^{l}\alpha_i\alpha_j y_i y_j\psi_\sigma(x_i,x_j)$$
$$\text{s.t.}\ \ 0\le\alpha_i\le\theta\ \ \text{and}\ \ \sum_{i=1}^{l}\alpha_i y_i=0 \quad (33)$$

where α is the Lagrange coefficients, θ is a constant that is used to penalize the training errors, and ψ_σ is the kernel function. To be an acceptable kernel, ψ_σ is the Gaussian kernel

$$\psi_\sigma(x^i,x^j)=\exp\left(-\frac{\|x_i-x_j\|^2}{2\sigma^2}\right) \quad (34)$$

where the norm "$\|\cdot\|$" is the Euclidean norm and $\sigma\in\mathbb{R}^+$ tunes the flexibility of the kernel. When the optimal solution of (33) is found, i.e., α_i, classification of a sample x is achieved by checking which side of the hyperplane it belongs to

$$y=\text{sgn}\left(\sum_{i=1}^{l}\alpha_i y_i\psi_\sigma(x_i,x)+b\right). \quad (35)$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NUMBER OF TRAINING AND TESTING SAMPLES IN THE PU

Class number	Class name	Training samples	Test samples
1	Asphalt	100	6531
2	Meadows	100	18549
3	Gravel	100	1999
4	Trees	100	2964
5	Metal sheets	100	1245
6	Bare soil	100	4929
7	Bitumen	100	1230
8	Bricks	100	3582
9	Shadows	100	847
	Total	900	41876

TABLE III
NUMBER OF TRAINING AND TESTING SAMPLES IN THE IP

Class number	Class name	Training samples	Test samples
1	Alfalfa	5	41
2	Corn-notill	143	1285
3	Corn-mintill	83	747
4	Corn	24	213
5	Grass-pasture	49	434
6	Grass-trees	73	657
7	Grass-pasture-mowed	3	25
8	Hay-windrowed	48	430
9	Oats	2	18
10	Soybean-notill	98	874
11	Soybean-mintill	246	2209
12	Soybean-clean	60	533
13	Wheat	21	184
14	Woods	127	1138
15	Buildings-Grass-Trees-Drives	39	347
16	Stone-Steel-Towers	10	83
	Total	1031	9218

IV. EXPERIMENTAL RESULTS AND ANALYSIS

To demonstrate the performance of the proposed method, three public databases, i.e., Pavia University (PU),[1] Indian Pines (IP),[2] and Salinas,[2] are utilized to conduct experiments. We study the classification performance of two different classifiers on the deep features extracted by the proposed DSAFDNet. To prevent the confusion of notation, we use the notation DSAFDNet_s and DSAFDNet_g to denote classification using the SVM classifier and GSL classifier, respectively.

According to the literature [26], the number of training samples and testing samples on the three datasets can be determined, as shown in Tables II–IV, respectively.

To quantitatively evaluate the classification performance of the proposed method, this article employs overall classification accuracy, Kappa, and accuracy under each category as evaluation measures [26]. All the listed results are averaged on five trails, and the best results are marked in bold.

A. Parameters Analysis and Setting

In this section, we will first discuss parameter settings that have a significant impact on the classification performance of the DTCSCNet_s and DTCSCNet_g, which include the number of network layer ℓ, image patch size, the number of decomposition, local region size of LBP, the parameters of SVM and GSL, in which local region size of LBP, the parameters of SVM and GSL remain the same as in literature [4], [25], respectively, where $\theta = 1024$ and $\sigma = 0.01$ in SVM, and $\delta_n = 0.1$, $\delta_{M-n} = 10^{-6}$, and $\vartheta = 0.3$ in GSL. As for t in PCA, we set $t = 3$.

Next, we will focus on analyzing the impact of parameters, including the number of network layer ℓ, image patch size ($w_0 \times h_0$), the number of the decomposition C on the performance of HSI classification, where the length and width of the image patch are selected to be the same, i.e., $w_0 = h_0$. In fact, the three parameters that are the number of network layer ℓ, image patch size, and the number of decomposition C all affect the classification accuracy and jointly determine the final classification effect.

Figs. 3 and 4 show the impact of the three parameters on the classification performance when SVM and GSL are used as classifiers, respectively. As can be seen from Fig. 3, when SVM is used as a classifier, the classification with too many network layers cannot achieve high accuracy. A network with less than five layers can obtain higher classification accuracy. On the contrary, the higher classification accuracy of DSAFDNet_s can be obtained when using a larger image patch (i.e., greater than 29) and decomposition number C (i.e., greater than 30). Weighting the computational cost and the classification accuracy, in the case of using SVM as a classifier, we set $\ell = 4$, $w_0 = 31$, and $C = 30$ for all three datasets, respectively.

It can be seen from Fig. 4 that when GSL is used as a classifier, the trends of the parameters are similar to those of SVM, but the values are slightly different. We set the parameters as $\ell = 5$, $w_0 = 27$, and $C = 35$ for all three datasets, respectively. In addition, for the settings of other parameters, such as the size $d = 9$ in the LBP algorithm, we kept the settings in the literature [27].

TABLE IV
NUMBER OF TRAINING AND TESTING SAMPLES IN THE SALINAS

Class number	Class name	Training samples	Test samples
1	Brocoli Green Weeds 1	100	1909
2	Brocoli Green Weeds 2	100	3926
3	Fallow	100	1876
4	Fallow Rough Plow	100	1294
5	Faloow Smooth	100	2578
6	Stubble	100	3859
7	Celery	100	3479
8	Grapes Untrained	100	11171
9	Soil Vinyard Develop	100	6103
10	Corn Senesced Green Weeds	100	3178
11	Lettuce Romaine 4wk	100	968
12	Lettuce Romaine 5wk	100	1827
13	Lettuce Romaine 6wk	100	816
14	Lettuce Romaine 7wk	100	970
15	Vinyard Untrained	100	7168
14	Vinyard Vertical Trellis	100	1707
	Total	1600	52529

B. Classification Results

In this section, we report the classification results of the proposed methods and the comparison results with other state-of-the-art methods. The comparison methods are mainly divided into two categories: methods similar in principle to our method, i.e., using classifiers, and end-to-end DL methods. For the first category, we selected stacked autoencoder (SAE) [28], SVM based on radial basis function (RBF-SVM)

[1]http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes#Pavia_University_scene

[2]http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

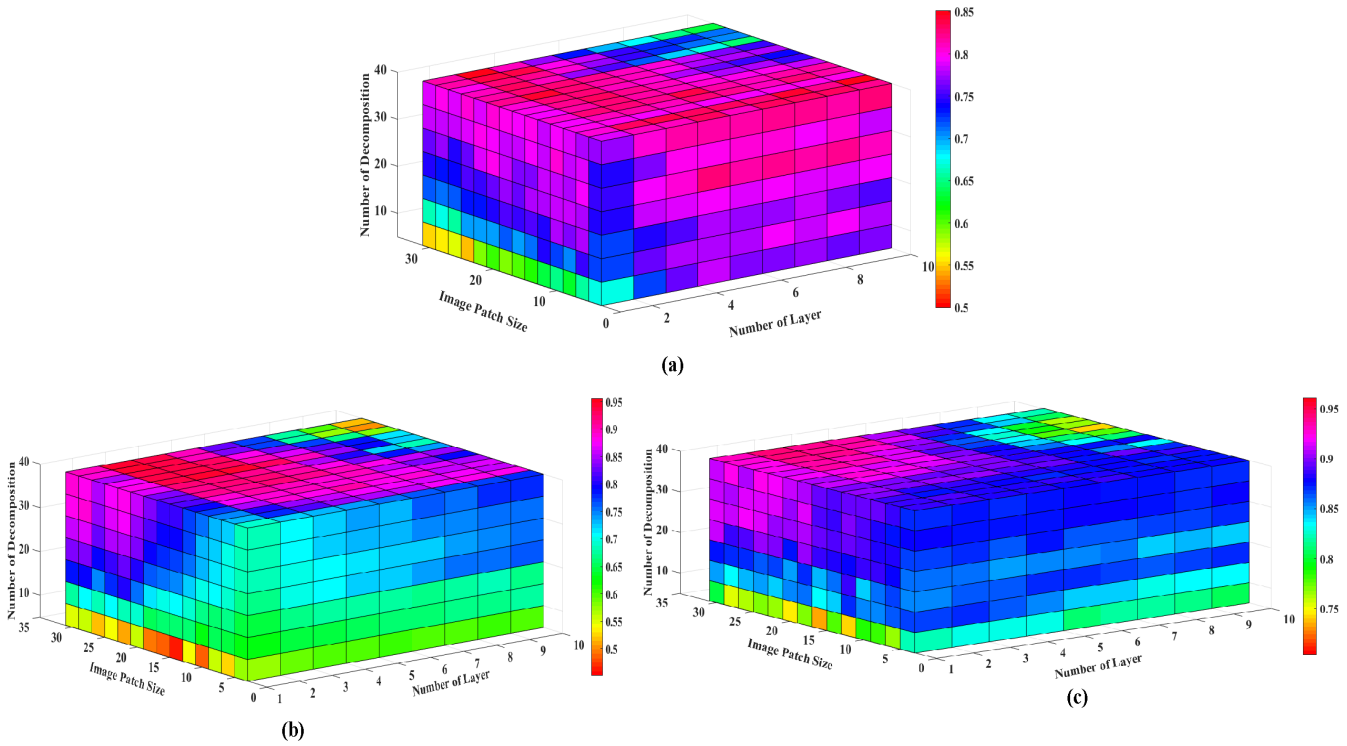

Fig. 3. Parameters analysis of DSAFDNet using classifier SVM (DSAFDNet_s) on three datasets with different decomposition number, image patch size, and layers. (a) PU. (b) IP. (c) Salinas.

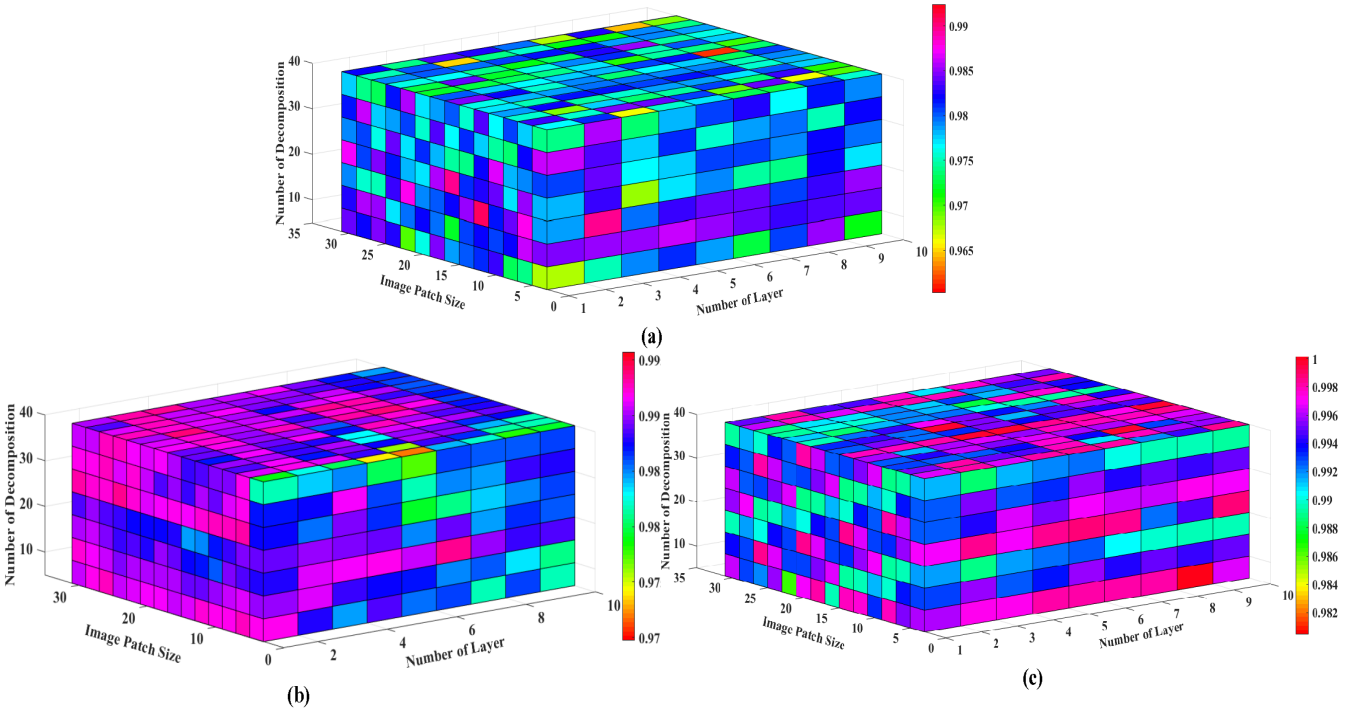

Fig. 4. Parameters analysis of DSAFDNet using classifier GSL (DSAFDNet_g) on three datasets with different decomposition number, image patch size, and layers. (a) PU. (b) IP. (c) Salinas.

in [25], GSL [29], random patches network (RPNet) [25], deep high-order tensor convolutional sparse coding network (DHTCSCNet) [27], spectral-spatial random patches network (SSRPNet) [4], and spectral-spatial fully convolutional network based on conditional random field (SSFCN-CRF) [30]. For the second category, we selected miniGCN [31], graph fusion network (GFNet) [32], and cross-attention spectral-spatial network (CASSN) [26]. All experiments are performed on three datasets. Numbers of training and test samples are given in Tables II–IV. The overall accuracy (OA) and kappa coefficient (κ) are utilized to quantitatively estimate the performance of different methods.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

TABLE V
QUANTITATIVE RESULTS COMPARISON AMONG THE DIFFERENT METHODS ON THE PU

Class	SAE	SVM	GSL	RPNet	DHTCSCNet	SSRPNet	SSFCN-CRF	miniGCN	GFNet	CASSN	DSAFDNet_s	DSAFDNet_g
1	92.25	52.61	86.03	92.31	83.69	96.15	91.29	71.43	93.37	**99.10**	82.02	98.61
2	91.19	63.33	90.26	96.74	91.31	98.27	95.15	94.79	96.52	96.56	89.31	**99.93**
3	89.39	54.23	43.19	97.27	46.91	80.08	95.15	83.29	85.28	**98.85**	96.10	96.87
4	98.87	93.63	59.11	97.91	80.26	89.26	97.64	92.07	97.54	**99.49**	94.60	96.17
5	99.97	95.69	76.06	**100**	95.94	98.17	98.96	99.20	**100**	99.84	**100**	**100**
6	93.28	78.22	41.30	97.72	76.76	94.42	97.95	77.26	95.47	98.11	95.23	**99.74**
7	96.36	84.39	42.81	98.73	50.55	87.19	95.37	96.26	92.86	99.92	**100**	99.59
8	96.39	61.25	60.20	**97.33**	82.43	88.07	95.62	74.62	89.90	94.92	94.53	95.24
9	99.80	99.80	99.88	99.88	97.12	91.43	99.53	**100**	**100**	**100**	91.74	99.76
OA (%)	92.97	67.26	66.45	96.54	82.16	94.12	95.30	86.90	94.92	97.58	90.70	**98.85**
$\kappa\times$ 100	90.55	59.37	58.45	95.42	76.32	92.25	93.79	82.56	93.28	96.79	87.86	**98.47**
Runtime (s)	374.23	**9.34**	123.65	35.71	211.31	680.49	4030.25	253.76	285.60	57.57	100.64	109.92

TABLE VI
QUANTITATIVE RESULTS COMPARISON AMONG THE DIFFERENT METHODS ON THE IP

Class	SAE	SVM	GSL	RPNet	DHTCSCNet	SSRPNet	SSFCN-CRF	miniGCN	GFNet	CASSN	DSAFDNet_s	DSAFDNet_g
1	**100**	84.55	**100**	79.67	90.84	94.00	90.24	24.39	**100**	**100**	**100**	**100**
2	82.86	68.02	59.34	94.89	76.91	98.92	74.63	58.83	86.39	93.23	**99.30**	98.21
3	94.10	66.67	53.82	94.07	84.12	97.49	75.64	58.90	82.43	92.37	96.52	**98.29**
4	92.63	71.67	49.33	86.38	83.61	97.88	84.04	47.42	82.14	93.90	**100**	**100**
5	97.15	88.48	90.61	96.24	89.09	99.22	69.12	89.63	92.86	98.16	98.16	**99.54**
6	98.60	87.32	72.56	99.24	81.11	97.67	82.19	95.74	98.80	99.24	96.50	**100**
7	98.75	77.33	61.69	89.33	83.09	92.67	68.00	4.00	50.00	**100**	**100**	**100**
8	99.79	**100**	98.81	99.92	81.55	**100**	99.53	97.44	**100**	99.30	**100**	**100**
9	**100**	81.48	93.94	53.70	93.06	96.67	22.22	0.00	**100**	88.89	94.44	**100**
10	90.29	84.06	76.77	94.93	87.65	98.03	79.18	70.82	83.75	93.59	94.74	**99.42**
11	84.39	88.00	72.28	98.40	75.47	98.86	88.55	61.61	87.71	98.14	98.87	**99.19**
12	93.93	56.85	46.70	92.31	84.82	96.41	56.66	66.04	94.34	94.18	97.00	**99.44**
13	99.64	70.11	86.77	99.28	92.24	99.08	94.02	95.11	**100**	98.91	88.04	**100**
14	96.78	96.34	95.36	99.41	95.20	99.39	92.97	88.84	97.67	99.74	99.82	**100**
15	85.51	60.42	50.41	93.85	89.82	99.43	68.01	42.07	71.88	97.41	92.51	**100**
16	**99.77**	77.11	92.45	79.12	94.49	89.42	93.98	95.18	88.89	98.80	96.39	98.78
OA (%)	90.11	80.95	72.22	96.24	82.23	98.45	81.62	70.38	89.76	96.55	97.96	**99.29**
$\kappa\times$ 100	85.56	78.21	68.11	95.09	79.46	98.23	79.01	66.60	88.30	96.07	97.67	**99.20**
Runtime (s)	242.7	**5.07**	63.21	14.22	77.46	71.63	1879.51	90.85	223.43	90.05	75.39	83.32

TABLE VII
QUANTITATIVE RESULTS COMPARISON AMONG THE DIFFERENT METHODS ON THE SALINAS

Class	SAE	SVM	GSL	RPNet	DHTCSCNet	SSRPNet	SSFCN-CRF	miniGCN	GFNet	CASSN	DSAFDNet_s	DSAFDNet_g
1	**100**	99.06	95.54	99.48	94.91	**100**	96.18	98.95	**100**	**100**	**100**	**100**
2	**100**	97.29	98.70	99.79	96.90	**100**	99.26	99.83	**100**	99.94	97.52	**100**
3	95.13	96.93	95.02	99.86	98.87	99.88	97.76	99.57	**100**	99.52	98.83	**100**
4	49.02	97.60	94.84	99.69	98.44	98.09	95.52	99.69	99.31	99.85	**99.92**	98.63
5	99.05	96.55	95.36	98.94	98.43	99.43	97.98	98.18	**100**	98.91	98.06	99.88
6	99.30	99.09	99.85	99.89	99.38	99.88	99.17	99.84	**100**	99.97	99.25	**100**
7	**100**	99.25	99.04	99.78	94.02	99.81	98.51	99.91	**100**	99.97	**100**	99.97
8	93.03	79.95	77.66	86.09	99.51	99.84	81.49	79.67	87.39	91.04	94.06	**99.98**
9	96.92	93.99	92.70	99.69	99.43	99.61	94.10	99.95	99.84	**99.97**	99.69	99.82
10	98.99	84.38	75.57	98.35	99.33	99.95	93.11	92.89	98.76	99.40	99.81	**100**
11	90.70	91.32	64.18	99.83	99.09	98.22	97.83	96.90	**100**	99.28	**100**	**100**
12	99.51	93.05	85.20	**100**	94.24	99.69	99.78	99.95	**100**	**100**	99.95	**100**
13	96.49	96.49	79.56	99.59	86.32	99.47	99.88	99.14	**100**	**100**	99.88	**100**
14	55.41	93.44	83.66	99.11	98.94	98.98	97.42	93.30	99.04	99.90	**100**	99.59
15	**99.95**	79.28	60.11	86.87	95.11	81.33	84.82	68.04	85.51	92.41	99.20	99.69
16	**100**	98.46	97.82	98.89	99.78	99.57	97.66	98.42	**100**	99.59	99.94	**100**
OA (%)	93.49	89.11	84.29	94.93	97.59	96.61	92.08	90.45	95.40	96.91	98.21	**99.88**
$\kappa\times$ 100	92.74	87.88	82.52	94.35	97.31	96.12	91.18	89.35	94.89	96.55	98.01	**99.70**
Runtime (s)	483.46	**21.20**	448.34	32.04	223.29	436.39	6140.17	495.81	545.09	151.07	303.71	476.33

The results of the quantitative evaluation are shown in Tables V–VII. From the results in Tables V–VII, it can be seen that the proposed DSAFDNet_g with classifier GSL generally outperforms other methods similar in principle to our method, such as SAE, SVM, GSL, RPNet, DHTCSCNet, SSRPNet, and SSFCN-CRF on three datasets. In addition, another version of our method, DSAFDNet_s with classifier SVM, achieves the third and second classification accuracy among all compared methods on the IP dataset and the Salinas dataset, respectively. For the comparison methods similar in principle to our method, RPNet and SSRPNet performed the best, where RPNet won the first, second, and third on PU, IP, and Salinas, respectively; SSRPNet won the third, first, and second on PU, IP, and Salinas, respectively. The OA of DSAFDNet_g outperforms RPNet by 2.3%, 3.0%, and 4.9% on the PU, IP, and Salinas datasets, respectively; the kappa coefficient (κ) of DSAFDNet_g outperforms RPNet by 3.0%, 4.1%, and 5.3% on the PU, IP, and Salinas datasets, respectively. Meanwhile, the OA of DSAFDNet_g outperforms SSRPNet by 4.7%, 0.8%, and 3.2% on the PU, IP, and Salinas datasets, respectively; the kappa coefficient (κ) of DSAFDNet_g outperforms SSRPNet by 6.2%, 0.9%, and 3.5% on the PU, IP, and Salinas datasets, respectively. For another version of our method (i.e., DSAFDNet_s), the OA of DSAFDNet_s outperforms RPNet by 1.7% and 3.2% on the IP and Salinas datasets, respectively; the kappa coefficient (κ) of DSAFDNet_s outperforms RPNet by 2.5% and 3.6% on the IP and Salinas datasets, respectively; DSAFDNet_s outperforms

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

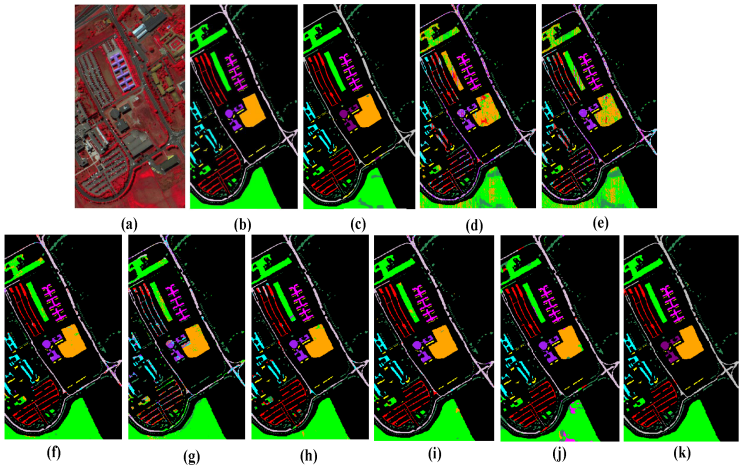

Fig. 5. Classification maps of different methods on PU. (a) False color map. (b) Real feature marker. (c) SAE (92.97%). (d) SVM (67.26%). (e) GSL (66.45%). (f) RPNet (96.54%). (g) DHTCSCNet (82.16%). (h) SSRPNet (94.12%). (i) SSFCN-CRF (95.30%). (j) DSAFDNet_s (90.70%). (k) DSAFDNet_g (98.85%).

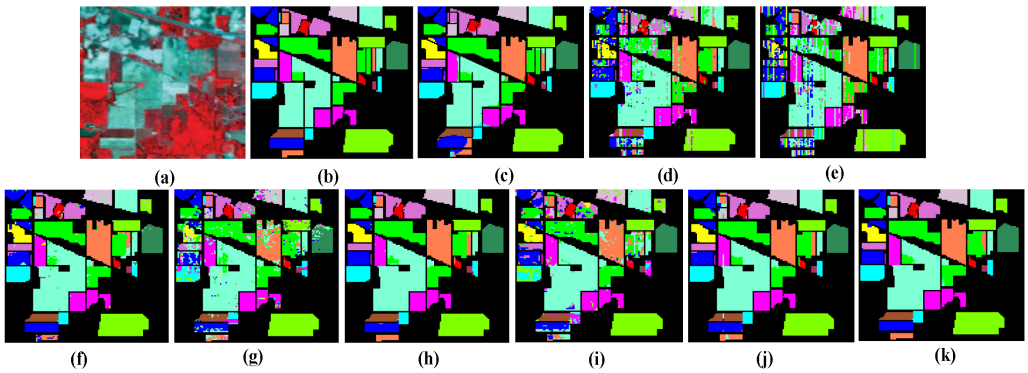

Fig. 6. Classification maps of different methods on IP. (a) False color map. (b) Real feature marker. (c) SAE (90.11%). (d) SVM (80.95%). (e) GSL (72.22%). (f) RPNet (96.24%). (g) DHTCSCNet (82.23%). (h) SSRPNet (98.45%). (i) SSFCN-CRF (81.62%). (j) DSAFDNet_s (97.96%). (k) DSAFDNet_g (99.29%).

SSRPNet by 1.6% on the Salinas dataset; the kappa coefficient (κ) of DSAFDNet_s outperforms SSRPNet by 1.8% on the Salinas dataset.

For the compared state-of-the-art end-to-end DL methods, CASSN achieved the highest classification accuracy. The OA of DSAFDNet_g is 1.2%, 2.7%, and 2.9% higher than CASSN on the PU, IP, and Salinas datasets, respectively; the kappa coefficient (κ) of DSAFDNet_g is 1.6%, 3.1%, and 3.1% higher than CASSN on the PU, IP, and Salinas datasets, respectively. In addition, another version DSAFDNet_s, whose OA is 1.4% and 1.3% higher than CASSN on the IP and Salinas datasets, respectively; the kappa coefficient (κ) is 1.6% and 1.4% higher than CASSN on the IP and Salinas datasets, respectively.

In addition, the visualization results of the classification maps in terms of the comparison methods with similar principles to our method are shown in Figs. 5–7. We can find that the visualization results are consistent with the analysis above. DSAFDNet_g and DSAFDNet_s obtained better classification maps, which are smoother and less noisy. Other comparison methods have too many noise points, and there are also a lot of misclassified pixels. Although RPNet and SSRPNet also obtained better classification maps among these comparison methods, compared with the proposed DSAFDNet_g and DSAFDNet_s, their classification maps have more misclassified pixels, which can also be concluded from their OA values.

Besides accuracy, running time is also an important factor for HSI classification task, especially in practical applications. One of the benefits of the proposed DSAFDNet_s and DSAFDNet_g is high efficiency, since we no longer need back propagation and fine-tuning from HSIs. In contrast, SSFCN-CRF, miniGCN, and GFNet all use the back propagation and fine-tuning process. It can be observed from Tables V–VII that SVM takes the least amount of time. In terms of running time, our methods take less time than SSFCN-CRF, miniGCN, and GFNet. CASSN uses back propagation and fine-tuning, but it takes less time than our method, because CASSN uses GPU acceleration during experiments, while our method is executed on the CPU (MATLAB). RPNet, DHTCSCNet, and SSRPNet do not use back propagation and fine-tuning. Although their feature extraction processes take less time than our method, these methods have lower accuracy. Our methods use dual-branch and LBP, which are time-consuming operations, and a better balance between accuracy and efficiency.

C. Analysis and Discussion

In this section, we investigate the effect of different training sample percentages on the proposed DSAFDNet_s and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

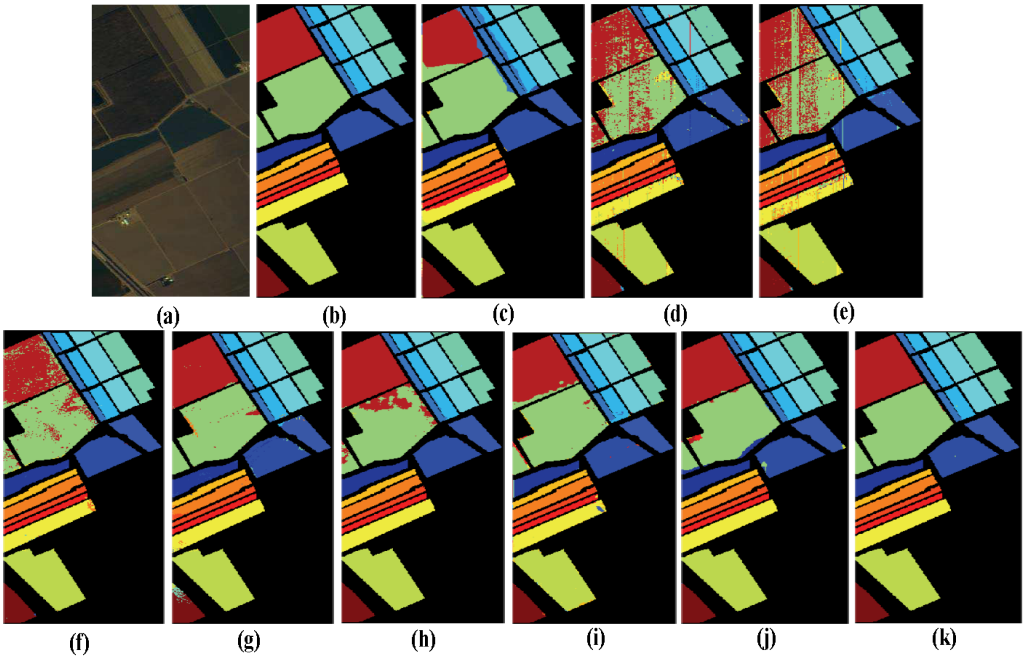

Fig. 7. Classification maps of different methods on Salinas. (a) False color map. (b) Real feature marker. (c) SAE (93.49%). (d) SVM (89.11%). (e) GSL (84.29%). (f) RPNet (94.93%). (g) DHTCSCNet (97.59%). (h) SSRPNet (96.61%). (i) SSFCN-CRF (92.08%). (j) DSAFDNet_s (98.21%). (k) DSAFDNet_g (99.88%).

TABLE VIII
QUANTITATIVE RESULTS COMPARISON AMONG THE DIFFERENT METHODS WITH FIVE TRAINING SAMPLES FOR EACH CLASS

	DHTCSCNet	SSRPNet	SSFCN-CRF	miniGCN	GFNet	CASSN	MDN	TSCSCN	DSAFDNet_s	DSAFDNet_g
PU	59.65	64.93	58.12	66.85	73.71	63.75	67.94	61.01	78.99	**79.52**
IP	63.71	63.52	62.21	60.16	64.56	62.49	63.02	61.71	67.97	**68.96**
Salinas	78.93	75.87	62.45	74.81	77.10	75.60	77.76	73.40	81.37	**83.82**

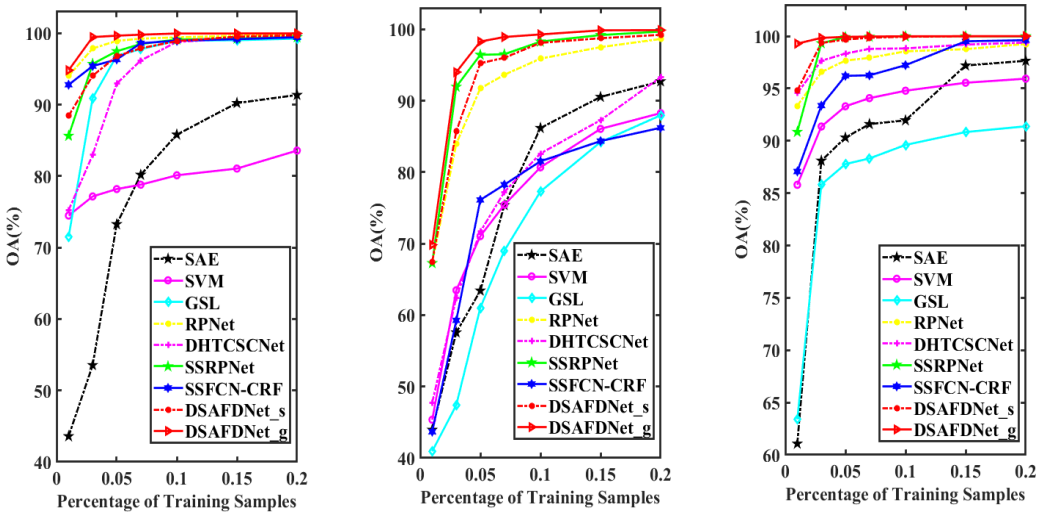

Fig. 8. Effects of training sample percentages on PU, IP, and Salinas datasets.

DSAFDNet_g. In addition, we also analyze the impact of dual branch and LBP on the performance of the proposed DSAFDNet_s and DSAFDNet_g.

1) Impact of Training Sample Percentages: Fig. 8 shows the effects of the training sample sizes on classification performance on the PU, IP, and Salinas datasets, respectively. On these three datasets, 1%, 3%, 5%, 7%, 10%, 15%, and 20% of the samples of each class are randomly selected for training, and the remaining samples are used for testing. It can be seen from Fig. 8 that classification accuracy of all the methods on three datasets steadily increases as the training size increases, and only when the training size is large enough, some comparison methods can obtain similar classifications to our method. However, the proposed DSAFDNet_g consistently performs best, especially when the training size is very small. The proposed DSAFDNet_s can also maintain the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

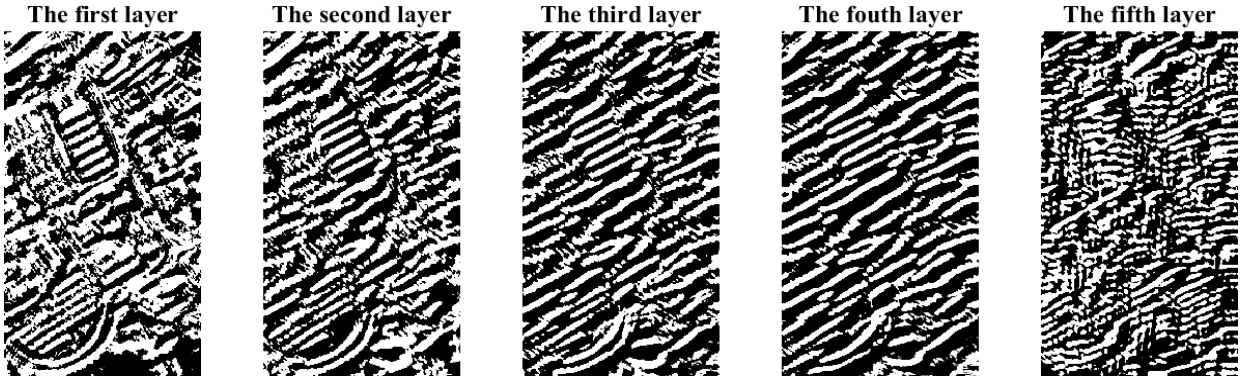

Fig. 9. Visualization of output feature map of the first five layers of SAFD network on PU dataset.

top three classification accuracy. Taking all the results into consideration, we can conclude that the proposed method has better generalization performance, especially when the training samples are limited.

2) Comparison With CNN-Based Few-Shot Learning Methods: The proposed method is compared with some state-of-the-art CNN-based few-shot learning methods. Table VIII shows the classification performance of different methods with five training samples for each class. These comparison methods are mainly divided into CNNs (i.e., SSRPNet, SSFCN-CRF), graph neural networks (i.e., miniGCN and GFNet), and other networks (attention network, CASSN). In addition, we compared with more recently proposed CNN-based methods (i.e., DHTCSCNet, multidomain network (MDN) [33], two-stage convolutional sparse coding network (TSCSCN) [34]), where DHTCSCNet is from the CNN method published in the year 2022, and MDN, TSCSCN are from the CNN method published in the year 2023. It can be seen from Table VIII that the proposed DSAFDNet_g obtains the best classification results with five training samples for each class, and another proposed method DSAFDNet_s also obtains the second highest classification accuracy. Compared with the recently published CNN-based methods DHTCSCNet, MDN, and TSCSCN, the proposed method DSAFDNet_g outperforms the best-performing method MDN on the PU dataset by 11%. It also outperforms the best-performing method, DHTCSCNet, by 5% and 4.8% on the IP and Salinas datasets, respectively.

3) Ablation Studies: The Impact of Dual Branch and LBP on Classification Performance: The proposed DSAFDNet_s and DSAFDNet_g contain two key components: the dual branch and LBP. The performance of both components is studied on three datasets. Classification results are listed in Table IX in which "✓" indicates that the corresponding method is selected, and "×" indicates that the corresponding method is not used. It can be seen from the results in Table IX that, without using LBP, the classification accuracy of DSAFDNet_s and DSAFDNet_g using the dual branch is higher than that of using the single branch. In the case of using the dual branch (or single branch), the proposed DSAFDNet_s and DSAFDNet_g can obviously improve the classification accuracy by using LBP. Moreover, the proposed DSAFDNet_s and DSAFDNet_g achieve higher classification performance only when dual branch and LBP are used together. In summary, this shows the effectiveness and reliability of the proposed DSAFDNet_s and DSAFDNet_g.

TABLE IX
ABLATION STUDIES OF DSAFDNET ON THREE DATASETS

		Single-branch	Dual-branch	LBP	OA (%)
DSAFDNet+SVM	PU	✓	×	×	85.20
		✓	×	✓	88.55
		×	✓	×	89.32
		×	✓	✓	**90.70**
	IP	✓	×	×	95.36
		✓	×	✓	97.65
		×	✓	×	96.24
		×	✓	✓	**97.96**
	Salinas	✓	×	×	94.49
		✓	×	✓	97.01
		×	✓	×	95.43
		×	✓	✓	**98.21**
DSAFDNet+GSL	PU	✓	×	×	91.68
		✓	×	✓	98.08
		×	✓	×	95.63
		×	✓	✓	**98.85**
	IP	✓	×	×	95.26
		✓	×	✓	98.95
		×	✓	×	98.68
		×	✓	✓	**99.29**
	Salinas	✓	×	×	94.45
		✓	×	✓	98.06
		×	✓	×	97.85
		×	✓	✓	**99.88**

4) Running Time (Training and Test Time) Comparisons: The training and test time of the proposed method are compared with more recently published CNN-based methods. The results are shown in Table X, where the recently published CNN-based methods in the table are DHTCSCNet (year 2022), MDN (year 2023), and TSCSCN (year 2023). It can be seen from Table X that the proposed DSAFDNet_s is the least in terms of total running time. Another version of the proposed method DSAFDNet_g also gets the top three total running time. For training time, the proposed methods (i.e., DSAFDNet_g and DSAFDNet_s) do not use backpropagation algorithms to fine-tune the network weights in the feature extraction stage, and only the time cost is generated by the classifier GSL or SVM in the training process. It can be seen from Table X that the proposed DSAFDNet_s is the least in terms of training time, and the proposed DSAFDNet_g also obtains the top three training time. However, SSFCN-CRF,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

TABLE X

RUNNING TIME (TRAINING AND TEST TIME) COMPARISON AMONG THE DIFFERENT METHODS. THE NOTATION "†" INDICATES THAT THE ALGORITHM HAS NO TRAINING PROCESS

		DHTCSCNet	SSRPNet	SSFCN-CRF	miniGCN	GFNet	CASSN	MDN	TSCSCN	DSAFDNet_s	DSAFDNet_g
PU	training time	†	†	2440.23	241.84	273.03	530.22	3125.34	†	†	†
	test time	290.64	102.82	356.47	11.92	12.57	4.35	15.67	708.92	98.74	173.91
	Total	290.64	102.82	2796.70	253.76	285.60	534.57	3141.01	708.92	**98.74**	173.91
IP	training time	†	†	175.11	82.49	214.37	188.33	1654.72	†	†	†
	test time	69.52	38.96	84.24	8.36	9.06	1.72	6.82	94.43	33.51	79.21
	Total	69.52	38.96	259.35	90.85	223.43	190.05	1661.54	94.43	**33.51**	79.21
Salinas	training time	†	†	1412.57	476.74	522.14	341.25	2865.23	†	†	†
	test time	518.14	177.60	231.09	19.07	22.95	9.82	12.96	530.61	111.11	265.60
	Total	518.14	177.60	1643.66	495.81	545.09	351.07	2878.19	530.61	**111.11**	265.60

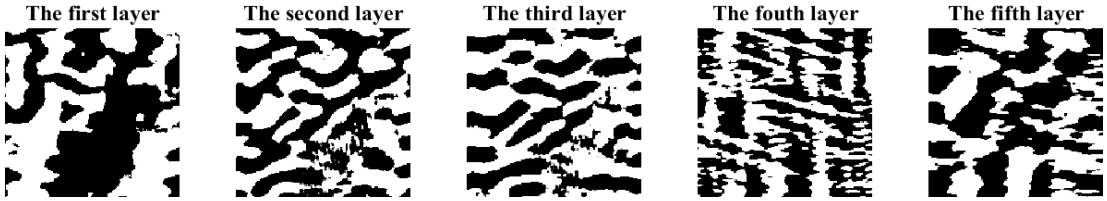

Fig. 10. Visualization of output feature map of the first five layers of SAFD network on IP dataset.

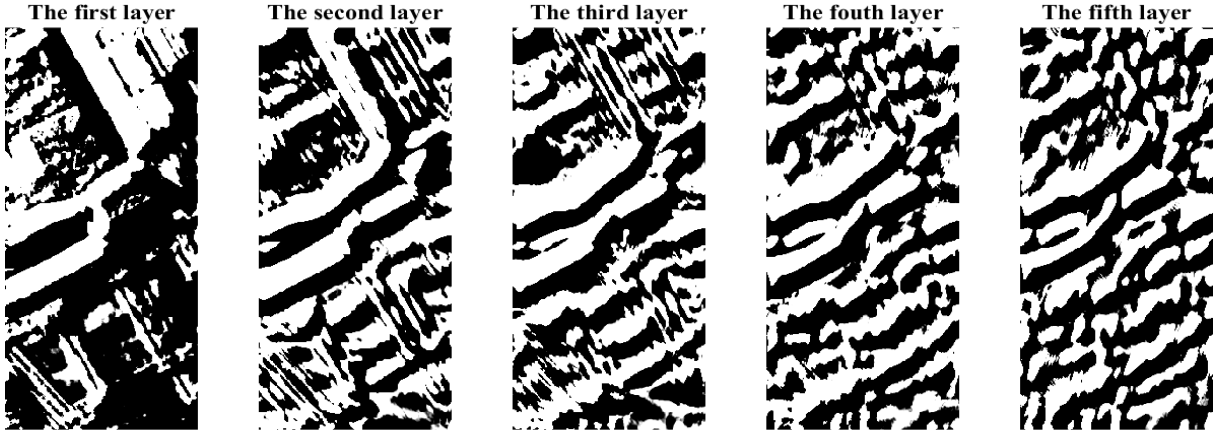

Fig. 11. Visualization of output feature map of the first five layers of SAFD network on Salinas dataset.

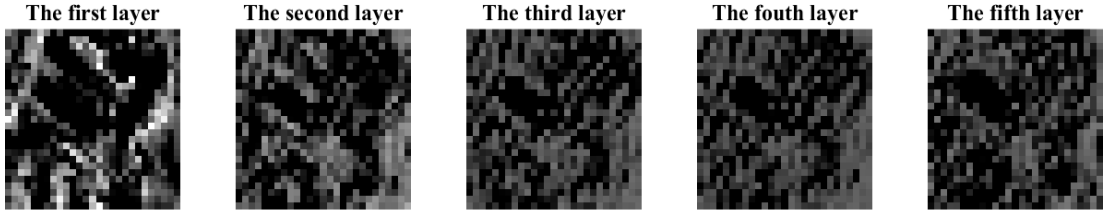

Fig. 12. Visualization of kernels (i.e., weights in SAFD) of the first five layers of SAFD network on PU dataset.

miniGCN, GFNet, CASSN, and MDN require training processes, and they all take more time than the proposed methods (i.e., DSAFDNet_g and DSAFDNet_s) in terms of training time.

5) Visualization of Output Feature Maps and Convolution Kernels: The output feature maps of the first five layers of the SAFD network are visualized, which are, respectively, shown in Figs. 9–11. It can be seen from Figs. 9–11. that as the number of the SAFD layers increases, the output features become more and more abstract. In addition, we also visualize the kernels (i.e., weights in SAFD) of the first five layers of the SAFD network, and the results of the three datasets are shown in Figs. 12–14. It should be pointed out here that we have only visualized one kernel and feature map per layer. It can be seen from Figs. 12–14 that the weights in SAFD and the kernels in CNN have similar functions, both of which are filtering and feature extraction. Structurally, the weights in SAFD and the kernels in CNN are also very similar. Therefore, each weight in SAFD (see Figs. 12–14) can be regarded as a kernel for filtering and image feature extraction. In addition, it can be

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

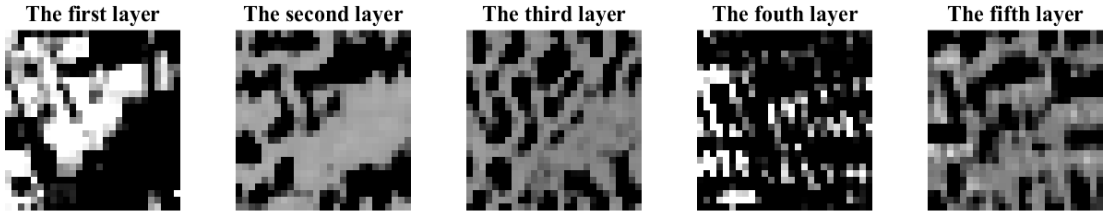

Fig. 13. Visualization of kernels (i.e., weights in SAFD) of the first five layers of SAFD network on IP dataset.

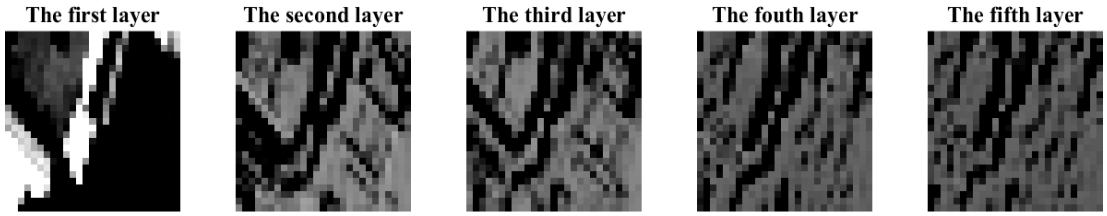

Fig. 14. Visualization of kernels (i.e., weights in SAFD) of the first five layers of SAFD network on Salinas dataset.

seen from Figs. 12–14 that the feature map of the proposed network is not very obvious in terms of abstract features. In general, the feature resolution near the input layer is high, while the feature resolution near the output layer is low. The size of different layers of filters and the size of features after convolution will affect the discrimination of abstract features. However, in this article, each layer of filter size and number, feature size is the same, mainly to ensure that the output feature dimension is consistent with the feature dimension to be classified, which simplifies the network and does not need to spend a lot of time adjusting parameters to achieve the optimal performance of the network.

V. Conclusion

In this article, we propose a deep SAFD network (DSAFD-Net), and the dual branch is used to extract features from HSIs using the real and imaginary parts of the complex convolution kernels trained by SAFD. During the classification process, we combine LBP, shallow, deep, spectral, and spatial features for classification. Features from multiple sources can help improve classification accuracy. Furthermore, we further investigate the classification performance of different classifiers on the extracted features. The popular SVM and GSL classifiers are used in this article. Experimental results show that DTCSCNet_g achieves higher classification accuracy among all compared methods, and another version of our method, DTCSCNet_s, also achieves good classification results.

In the future, we will continue to study efficient deep feature extractors and explore fast optimization solvers.

References

[1] H. Yu, H. Zhang, Y. Liu, K. Zheng, Z. Xu, and C. Xiao, "Dual-channel convolution network with image-based global learning framework for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[2] C. Yu, R. Han, M. Song, C. Liu, and C.-I. Chang, "Feedback attention-based dense CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501916.

[3] H. Yu, Z. Xu, K. Zheng, D. Hong, H. Yang, and M. Song, "MSTNet: A multilevel spectral–spatial transformer network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5532513.

[4] C. Cheng, H. Li, J. Peng, W. Cui, and L. Zhang, "Hyperspectral image classification via spectral–spatial random patches network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 4753–4764, 2021.

[5] H. Li, Y. Song, and C. L. P. Chen, "Hyperspectral image classification based on multiscale spatial information fusion," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 9, pp. 5302–5312, Sep. 2017.

[6] Z. Ye, H. Li, Y. Song, J. A. Benediktsson, and Y. Y. Tang, "Hyperspectral image classification using principal components-based smooth ordering and multiple 1-D interpolation," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 1199–1209, Feb. 2017.

[7] F. Gao, Q. Wang, J. Dong, and Q. Xu, "Spectral and spatial classification of hyperspectral images based on random multi-graphs," *Remote. Sens.*, vol. 10, no. 8, pp. 1271–1290, Jul. 2018.

[8] Q. Hao, B. Sun, S. Li, M. M. Crawford, and X. Kang, "Curvature filters-based multiscale feature extraction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5507916.

[9] L. Fang, W. Zhao, N. He, and J. Zhu, "Multiscale CNNs ensemble based self-learning for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 9, pp. 1593–1597, Sep. 2020.

[10] Y. Jiang, Y. Li, and H. Zhang, "Hyperspectral image classification based on 3-D separable ResNet and transfer learning," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 12, pp. 1949–1953, Dec. 2019.

[11] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[12] T. Li, X. Zhang, S. Zhang, and L. Wang, "Self-supervised learning with a dual-branch ResNet for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[13] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[15] S. Hao, W. Wang, Y. Ye, T. Nie, and L. Bruzzone, "Two-stream deep architecture for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 4, pp. 2349–2361, Apr. 2018.

[16] X. Li, M. Ding, and A. Pižurica, "Deep feature fusion via two-stream convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2615–2629, Apr. 2020.

[17] J. Feng et al., "Attention multibranch convolutional neural network for hyperspectral image classification based on adaptive region search," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5054–5070, Jun. 2021.

[18] J. Wang, S. Guo, R. Huang, L. Li, X. Zhang, and L. Jiao, "Dual-channel capsule generation adversarial network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501016.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.

[19] T. Qian, L. Zhang, and Z. Li, "Algorithm of adaptive Fourier decomposition," *IEEE Trans. Signal Process.*, vol. 59, no. 12, pp. 5899–5906, Dec. 2011.
[20] T. Qian and Y.-B. Wang, "Adaptive Fourier series—A variation of greedy algorithm," *Adv. Comput. Math.*, vol. 34, no. 3, pp. 279–293, Apr. 2011.
[21] L. Dai, L. Zhang, and H. Li, "Image compression using stochastic-AFD based multisignal sparse representation," *IEEE Trans. Image Process.*, vol. 31, pp. 5317–5331, 2022.
[22] L. Dai and L. Zhang, "A joint spatiotemporal video compression based on stochastic adaptive Fourier decomposition," *IEEE Signal Process. Lett.*, vol. 29, pp. 1531–1535, 2022.
[23] C. Tan, L. Zhang, T. Qian, S. Brás, and A. J. Pinho, "Statistical *n*-best AFD-based sparse representation for ECG biometric identification," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–13, 2021.
[24] T. Qian, "Sparse representations of random signals," *Math. Methods Appl. Sci.*, vol. 45, no. 8, pp. 4210–4230, May 2022.
[25] Y. Xu, B. Du, F. Zhang, and L. Zhang, "Hyperspectral image classification via a random patches network," *ISPRS J. Photogramm. Remote Sens.*, vol. 142, pp. 344–357, Aug. 2018.
[26] K. Yang, H. Sun, C. Zou, and X. Lu, "Cross-attention spectral–spatial network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518714.
[27] C. Cheng, H. Li, J. Peng, W. Cui, and L. Zhang, "Deep high-order tensor convolutional sparse coding for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5520611.
[28] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.
[29] L. Ma, A. Ma, C. Ju, and X. Li, "Graph-based semi-supervised learning for spectral–spatial hyperspectral image classification," *Pattern Recognit. Lett.*, vol. 83, pp. 133–142, Nov. 2016.
[30] Y. Xu, B. Du, and L. Zhang, "Beyond the patchwise classification: Spectral–spatial fully convolutional networks for hyperspectral image classification," *IEEE Trans. Big Data*, vol. 6, no. 3, pp. 492–506, Sep. 2020.
[31] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.
[32] X. Du, X. Zheng, X. Lu, and A. A. Doudkin, "Multisource remote sensing data classification with graph fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10062–10072, Dec. 2021.
[33] J. Wang, J. Zhou, and X. Liu, "Toward universal representation learning for multidomain hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5508116.
[34] C. Cheng, J. Peng, and W. Cui, "A two-stage convolutional sparse coding network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

Chunbo Cheng received the B.Sc. degree in mathematics and applied mathematics from Yangtze University, Jingzhou, China, in 2015, and the Ph.D. degree from the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan, China, in 2020.

He is currently a Lecturer with the College of Mathematical and Physical Sciences, Hubei Polytechnic University, Huangshi, China. His research interests include pattern recognition, machine learning, and computer vision.

Liming Zhang (Member, IEEE) received the M.S. degree in signal processing from the Nanjing University of Science and Technology, Nanjing, China, in 1990, and the Ph.D. degree in image processing from the University of New England, Armidale, NSW, Australia, in 2002.

She is currently an Assistant Professor with the Faculty of Science and Technology, University of Macau, Macau, China. Her research interests include signal processing, image processing, and computer vision.

Hong Li (Member, IEEE) received the M.Sc. degree in mathematics and the Ph.D. degree in pattern recognition and intelligence control from the Huazhong University of Science and Technology, Wuhan, China, in 1986 and 1999, respectively.

She is currently a Professor with the School of Mathematics and Statistics, Huazhong University of Science and Technology. Her research interests include approximation theory, wavelet analysis, learning theory, neural networks, signal processing, and pattern recognition.

Wenjing Cui received the B.S. degree from Henan Normal University, Xinxiang, China, in 2015, and the M.S. degree from the Wuhan University of Technology, Wuhan, China, in 2018.

She is currently an Assistant with the College of Mathematical and Physical Sciences, Hubei Polytechnic University, Huangshi, China. Her research interests include machine learning and econometric analysis.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 06:28:14 UTC from IEEE Xplore. Restrictions apply.